

Imaging project report

Computing methods for experimental physics and data analysis

Linda Biasci

1 Project description

The Paintings Damage Classification project aims to implement a classifier to distinguish between two diagnostic categories, developing two different pipelines:

- traditional approach: image processing and handcrafted feature extraction
- deep learning approach: feature extraction with a pre-trained deep neural network

Extracted features are used to train a machine learning classifier: its performance is evaluated and results obtained from the two pipelines are compared.

1.1 Dataset

The chosen dataset is Kaggle's "AI_for_Art_Restoration_2": a collection of artworks and pictures classified in "damaged" (positive class) and "undamaged" (negative class). This dataset consists of two subdirectories: `paired_dataset_art`, which contains pairs of correspondent damaged and undamaged paintings, and `unpaired_dataset_art`, which contains generic artworks divided into the damaged and undamaged folders.

2 Project workflow

The dataset was preliminarily checked: in the `paired_dataset` folder, duplicates were deleted and couples of images were renamed, so to obtain a structured correspondence among each pair of damaged and undamaged pictures. Via the MATLAB script `jpg_converter.m`, all images were converted to a common format and some of them were identified as corrupted files and deleted.

2.1 Image preprocessing

MATLAB was chosen for this initial stage because of its specific toolboxes, which provide dedicated functions for image processing and feature extraction. In the first part of `manual_feature_extractor.m`, all images are converted to RGB-double format and resized to 224x224 pixels, preserving their aspect and proportions through a zero-padding. Processed images are saved in a new directory, keeping the original dataset's folders structure.

2.2 Handcrafted feature extraction

Developing a MATLAB custom function, in the second part of `manual_feature_extractor.m` the following features are evaluated on the `paired_dataset`:

- seven texture features (grey-scaled images): contrast, homogeneity, energy and correlation from the Grey-Level Co-occurrence Matrix (surface damage detection), energy and entropy from the Local Binary Pattern algorithm (local measurements of textures), global entropy (Shannon entropy on pixels)
- an edge feature (grey-scaled images): manual Max Pooling on density of margins (ratio between the amount of edge pixels, picked out with Canny algorithm, and the total amount of pixels)
- a sharpness feature (grey-scaled images): manual Max Pooling on laplacian variance (Laplace image filtering)
- three Fourier spectral features (grey-scaled images): manual Max Pooling on spectral variance, ratio between high frequency components' energy and total spectral energy, normalised distance from the spectrum's centroid
- nine colour features: mean intensity, standard deviation and skewness on the three separated RGB channels

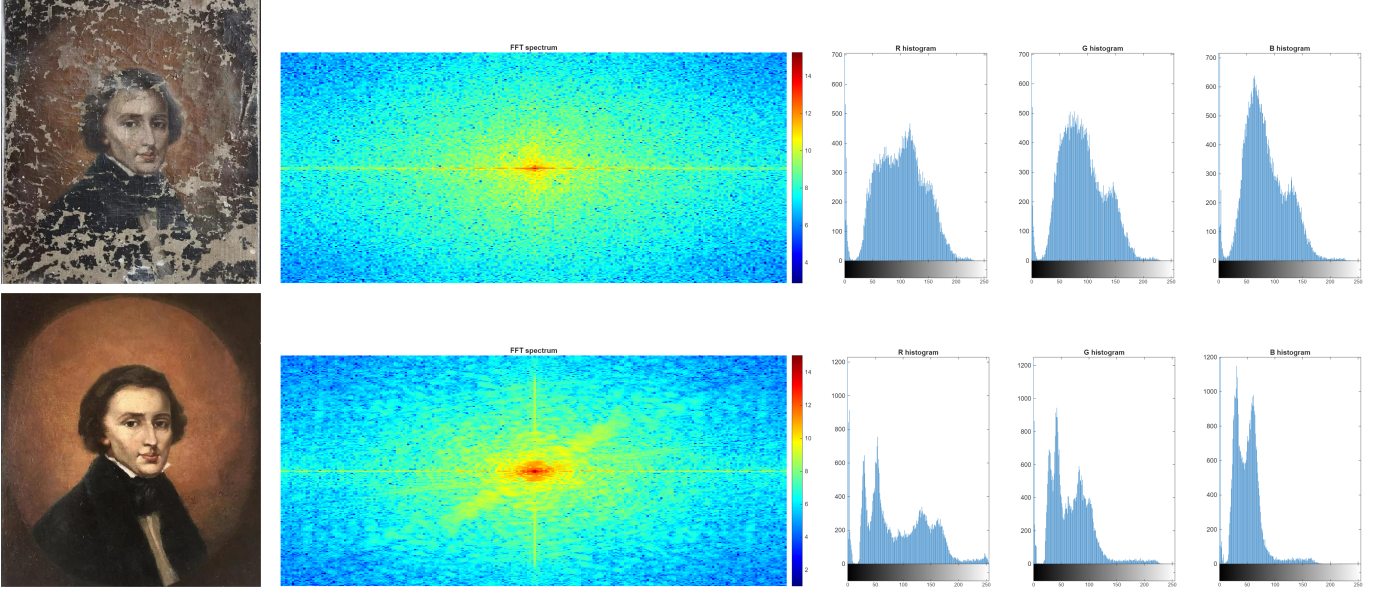
In order to identify the most diagnostic features, statistical analyses are performed on a table of differences between features extracted from the `damaged` and `undamaged` paired images (referred to as difference features):

- analysis of correlation matrix
- Lilliefors normality test to check if extracted features are normally distributed

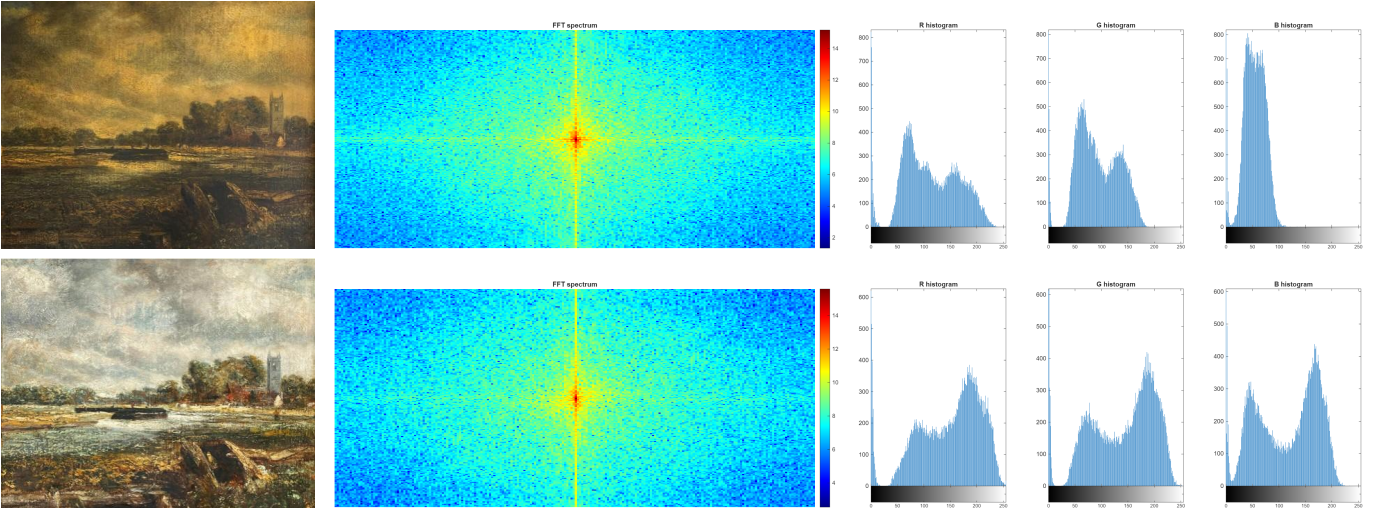
- Wilcoxon test to evaluate the statistical significance of non-normally distributed features
- visualisation of Fourier spectra, colour histograms, boxplots

Results: mean intensity, standard deviation and skewness are averaged over the three RGB channels due to correlation; contrast, homogeneity, energy from the GLCM, and laplacian variance are neglected mostly because of irrelevance; LBP energy and distance from spectral centroid are neglected since being, respectively, anti-correlated to and less significant than LBP entropy, and correlated to and less significant than spectral energy ratio.

The nine selected features are therefore extracted from `unpaired_dataset`, and results are exported in a `.csv` file.



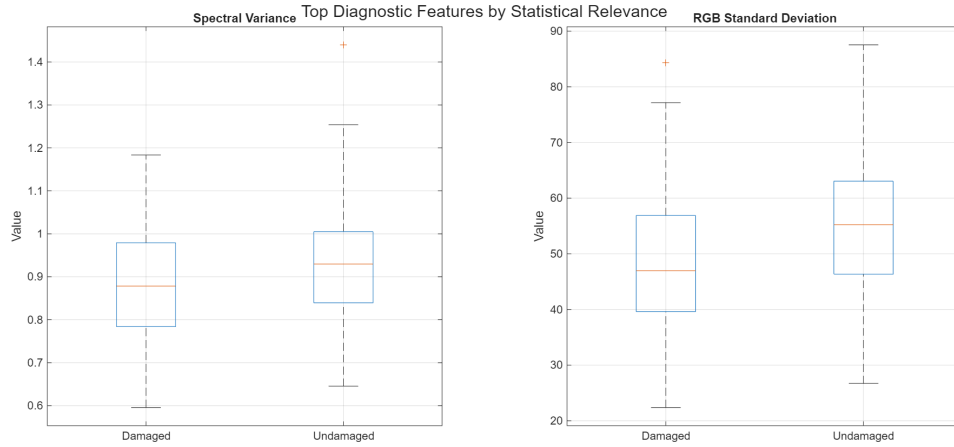
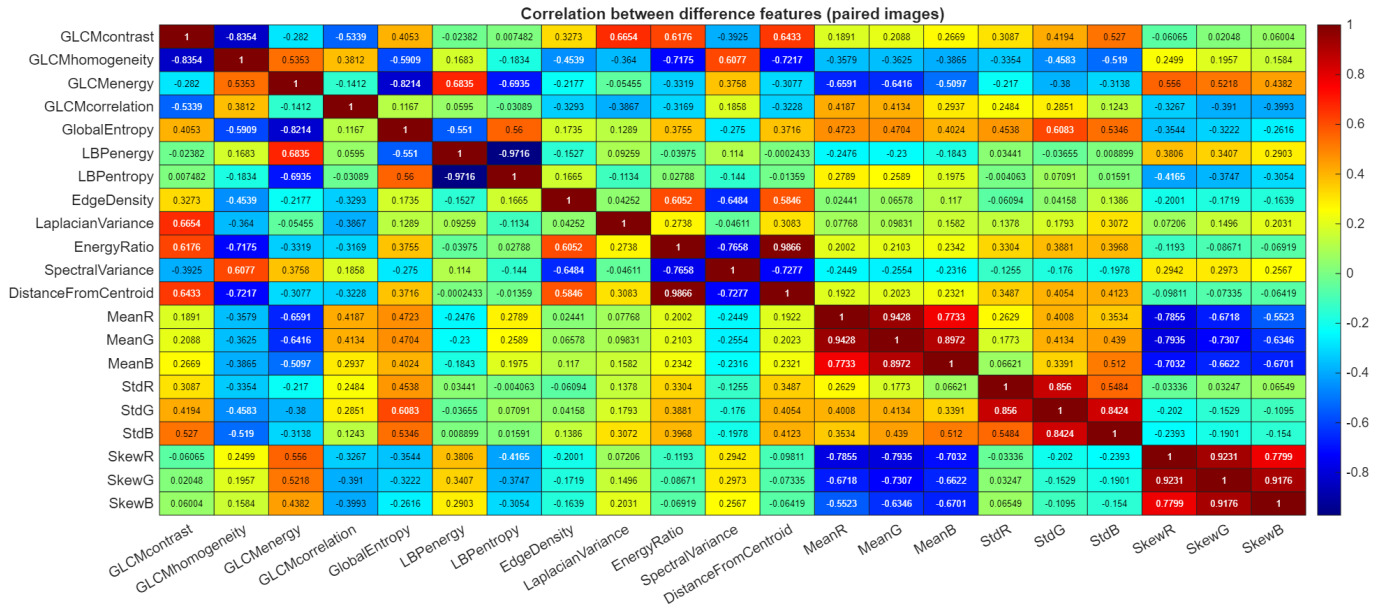
Comparison between damaged and restored portait, with correspondent Fourier spectrum and RGB histograms



Comparison between damaged and restored landscape, with correspondent Fourier spectrum and RGB histograms

2.3 Deep learning feature extraction

Python was chosen for deep learning and machine learning stages, as it provides specific and efficient libraries such as Keras, Scikit-learn and PyTorch. The deep learning pipeline is implemented using ResNet50, a convolutional neural network structured in 50 layers with skip connections, pre-trained on ImageNet, a database of more than a million images. Transfer learning is developed in `cnn_feature_extractor.py` by removing the last fully connected layer, originally used for image classification. The CNN can therefore be used as a feature extractor on the processed `unpaired_dataset`: the obtained output is a vector of 2048 deep features (instead of multi-class probabilities). Being the dataset relatively small, Principal Component Analysis is applied to the CNN's output vector, reducing its dimensionality to 50 features. Results are exported in a `.csv` file.



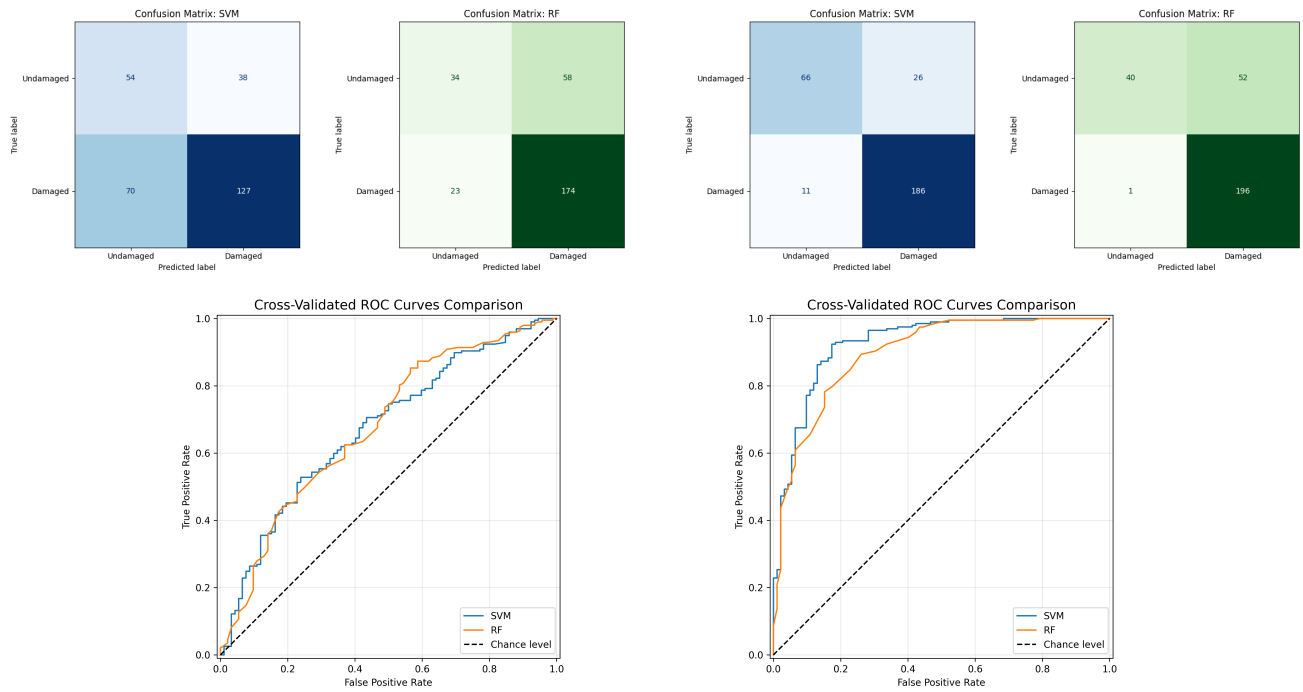
2.4 Machine learning classification

In the `ml_classifier.py` program, a Support Vector Machine and a Random Forest algorithm are implemented and their performances are compared. Because of the limited dataset available and being the two diagnostic categories not properly balanced, a Stratified K-Fold Cross Validation is employed instead of a train and test set division, and the `class_weight` parameter is set to “balanced” in both models. Metrics used to evaluate SVM and RF performances for both pipelines: accuracy, precision, recall (particularly relevant), F1 score, area under the ROC curve; ROC curves are also plotted, as well as confusion matrices (for both models in both pipelines).

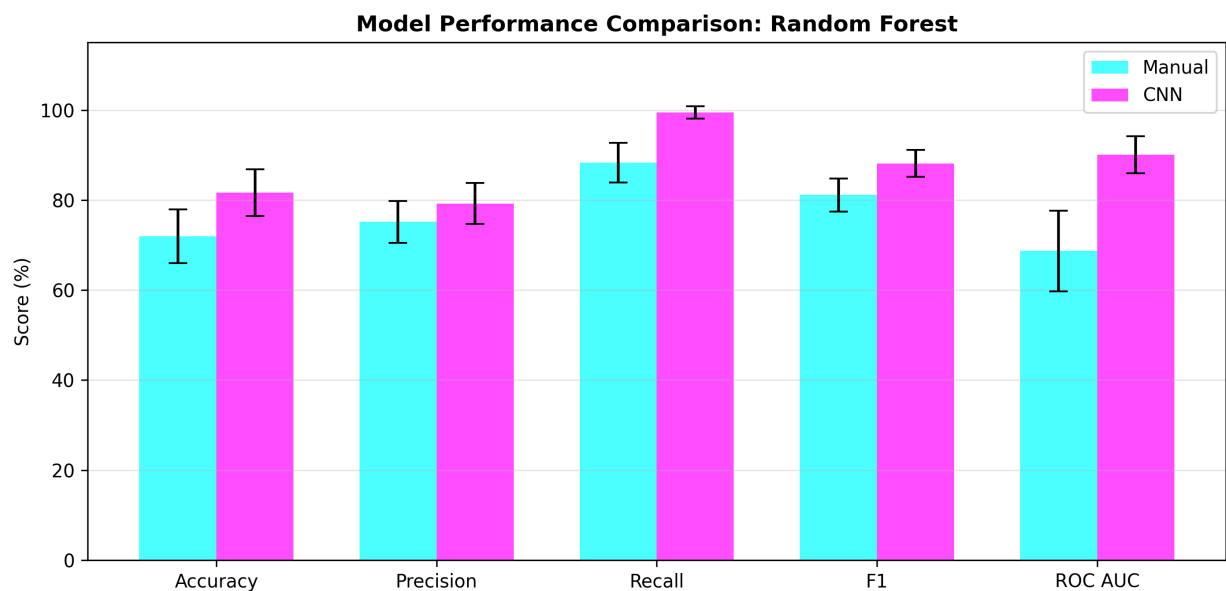
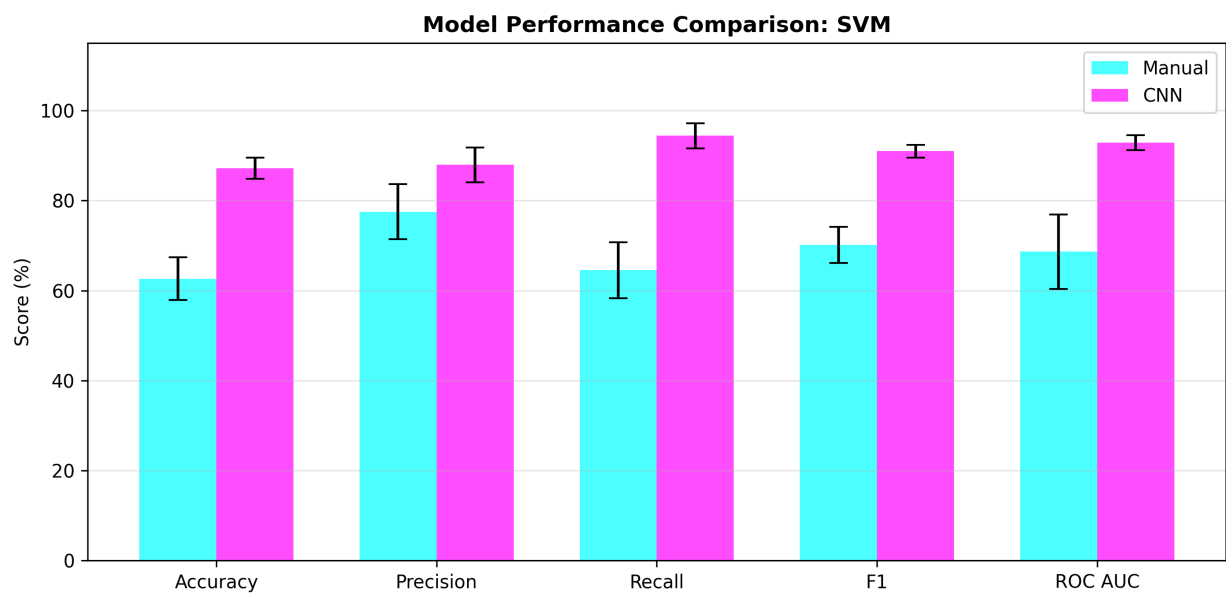
Results: in manual feature extraction the Random Forest model shows a better performance, being characterised by a higher recall and a smaller amount of false negatives; in deep feature extraction both the Support Vector Machine and the Random Forest models seem to be efficiently working.

3 Results

Metrics’ values (plotted via `pipelines_comparison.py`) and confusion matrices point out the better performance of CNN feature extraction with respect to manual feature extraction. These results might be due to a larger amount of features that are identified in the deep learning approach, extrapolating hidden information which could be even more significant (diagnostic) than the manually extracted one. Moreover, some of the handcrafted features could be too rough, being averaged over the whole picture: damage information might be concentrated just in a small, localised part of the painting, so diluting it over the whole might lessen its relevance, hence usefulness in classification. Nevertheless, using a DNN means much less interpretability: manually extracted features are explainable and follow statistics and logic, whereas deep features - despite their efficiency - appear as numbers coming out of a black box (none of them can be explicitly related, e.g., to surface damage rather than pigment degradation and so on).



Comparison between manual and deep feature extraction: confusion matrices and Receiver Operating Characteristic curves for Support Vector Machine and Random Forest models



Plots of metrics: mean values and standard deviations

4 Conclusions

Comparison between classification performances proves that CNN feature extraction is more reliable than manual feature extraction when it comes to distinguish between a damaged and an undamaged picture, always achieving higher metrics' values, hence being a more robust model. Despite that, the deep learning approach's main disadvantage is that such features cannot be described in terms of physical properties, so they might be unhelpful when restoration is required. Overall, DNN could be useful for a preliminary diagnostic classification of artworks, but a traditional analysis should be then carried out to formulate a restoration intervention.

Links

- [1] Raw dataset [here](#)
- [2] Preprocessed dataset (to be used as `manual_feature_extractor.m` input) [here](#)
- [3] More information available in the project's repository, [here](#)